# Automated Formal Equivalency Checking for FPGA Tools

Christopher R. Clark, William Stuckey, Nathan Braswell
Georgia Tech Research Institute
Georgia Institute of Technology, Atlanta, GA
{chris.clark, william.stuckey, nathan.braswell}@gtri.gatech.edu

Travis Haroldsen, Ting-Yuan Sung, Osaze Shears
Information Sciences Institute
University of Southern California, Arlington, VA
{tharoldsen, tsung, oshears}@isi.edu

***Abstract*—EDA tools are necessary to translate HDL source code to an FPGA implementation netlist. However, these tools can introduce undesired functional changes to the design, which may be the result of bugs in optimization algorithms or malicious modifications to the software. Formal equivalency checking (FEC) is an effective approach to address these concerns, but the difficulty and high level of effort required to apply this technique are impediments to widespread adoption. In this paper, we present a software framework that automates the configuration and execution of FEC tools. Automated analysis of logs and netlists produced by EDA tools is used to identify design optimizations and generate corresponding compare point mapping directives, or hints, that are required for effective and efficient formal analysis. The developed framework supports multiple commercial and open-source EDA and verification tools and easily integrates with existing build flows.**

***Keywords—FPGA; formal verification; equivalency checking***

## I. Background

Hardware designers rely on third-party EDA tools to translate their HDL source code of a design to an implemented circuit for an FPGA. However, the algorithms in these tools—synthesis, placement, routing, and timing optimization—could introduce undesired functional changes to the design, which may be the result of bugs in the algorithm implementation or malicious modifications to the software (e.g., to insert Trojan horse, backdoor, or timebomb functionality). Detection of these types of changes requires testing the netlist generated at the end of each EDA processing step. One testing approach commonly employed is functional simulation of the netlist. A major drawback of this approach is that it will only detect functional design changes that cause an observable discrepancy in the user-created suite of testbench stimulus generation and output checker modules, which is especially unlikely for malicious insertions due to their intentional stealthiness and long timeframes for activation.

A more thorough approach for detection of undesired functional changes introduced by EDA tools is Formal Equivalence Checking (FEC), which is a type of formal verification that uses mathematics to prove or disprove functional equivalence between two design representations. The procedure for using FEC to verify a synthesis operation is shown in Fig. 1 and comprises translating both the design's behavioral HDL source code and the post-synthesis netlist into formal models that specify their complete functional behavior using logical formal property statements. The models are respectively called the reference model and the implementation model. Finally, a formal analysis tool ingests both formal models and attempts to prove that all corresponding outputs of the reference and implementation models are identical when all possible value sequences are applied to the models' corresponding inputs. The possible FEC results are equivalent, not equivalent, or inconclusive.

FEC analysis involves evaluating equivalency at defined compare points, which are mappings between locations in each design representation that are expected to be identical. For example, each of the top-level output ports is a trivially-identified compare point. The most straightforward FEC problem consists of the top-level input ports as variables and the top-level output ports as compare points, however, the search-space complexity leads to impractical runtimes to solve this problem for large designs. So, it is necessary to define compare points between the top-level inputs and outputs to break the problem down into smaller independent sub-problems.

The accuracy and sufficiency of the compare point definitions is the most critical aspect of applying FEC. Where there are minimal changes between the HDL source code and the post-synthesis netlist, existing tools are able to use signal names to automate some compare point mappings, but this process is complicated by changes made by EDA tools that add, remove, reposition, and rename both logic and registers. In practical settings where optimization algorithms are employed for area, performance, and/or power, manual user-definition of compare points is required for effective and efficient FEC analysis. This can be a significant effort requiring specialized skills, making it a major impediment to adoption of this technique.

Our work seeks to reduce the time, effort, and skill level required to prove equivalence between the source HDL and the EDA-implemented netlist. We support use of FEC tools using both Logic Equivalency Checking (LEC) that only compares the combinational logic cones of the designs (i.e., the logic between each register or port is evaluated independently), and Sequential Equivalency Checking (SEC) that compares both combinational and sequential behavior of the designs thus avoiding the requirement for a complete set of register pin mappings but substantially increasing the complexity of the formal analysis.

The FEC procedure described above has some inherent requirements and assumptions that our tool is also limited by. First, it can only be applied when source code is available to be

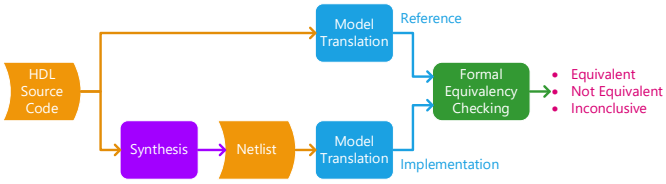


Fig. 1. Procedure for Formal Equivalency Checking of synthesis operation

translated into the reference model, which prohibits verification of third-party IP (3PIP) that is only available in encrypted or post-synthesis forms. Second, it only performs equivalency checking between the source HDL and the EDA-implemented netlist and thus cannot not find errors or malicious code already present in the source. Last, it assumes that the two formal model translation operations as well as the FEC analysis operation are correct; however, just like the EDA tools being checked, these are also software tools that are subject to coding errors and malicious modifications. Sec II discusses an approach to provide assurances of the correctness of the formal method implementations through cross-checking.

## II. FPGA EDA Tool Verification Framework

We have developed a framework that enables formal verification of the full FPGA EDA tool flow including synthesis, placement, and routing. Specifically, formal equivalency checking (FEC) is used to mathematically prove that the optimizations and other transformations performed by EDA tools have not altered the functional behavior specified by the user's HDL source code. The objective of our framework is to provide this assurance in the trustworthiness of the EDA tools with minimal effort from a designer. It integrates easily with multiple standard tool flows and requires no manual steps to configure and execute formal verification. The framework consists of a Python library as well as a self-contained Python application using this library that enable multiple usage models.

### A. FPGA Build Flow Integration

The primary use case is to add FEC to an existing FPGA build flow, as illustrated in Fig. 2. This is currently supported for Xilinx Vivado (project and non-project modes using either GUI or batch execution) as well as Synopsys Synplify (GUI or batch modes). To enable FEC, the user only needs to configure the settings provided by these tools to specify a TCL script to be launched at the end of a synthesis or implementation step. After pointing the EDA tool to our TCL script, all subsequent builds will call our automated FEC verification functions that analyze the output products from the EDA tool and generate the necessary files to configure and execute FEC analysis. We have implemented multiple automated FEC methodologies, which are detailed in Sec III. The tight integration includes the FEC tool output in the EDA tool's logs and generates warning and error messages in the same format and locations as the native application.

### B. EDA and FEC Verification Tool Control

Our framework provides a flexible mechanism for building and verifying designs for FPGAs. A generic, tool-agnostic API is available for each step in the process (i.e., synthesis, place and route, and FEC analysis). We have leveraged and extended the open-source EDAlize library [1] to generate and execute TCL scripts for a variety of tools. Fig. 3 shows how open-source, COTS, and FPGA vendor tools can be used in different combinations to sequence the steps for building and verifying a design. An interesting aspect is that multiple tools can be run for each step. In the case of FEC tools, this provides multiple independent verifications for increased assurance in the correctness of the formal analysis implementations. In the case of synthesis tools, this could be used to detect any functional differences in the output of different versions (indicating potential bugs) or even different instances of the same tool version (indicating potential binary corruption or tampering).

## III. Automated FEC Verification Methodologies

We have developed fully-automated FEC verification methodologies that process the output products from an EDA tool to extract optimization information, create corresponding compare point mapping hints and configuration files, and then generate and execute scripts for FEC analysis. The results of the equivalency check are extracted and returned to the EDA tool for presentation. This process is illustrated in Fig. 4.

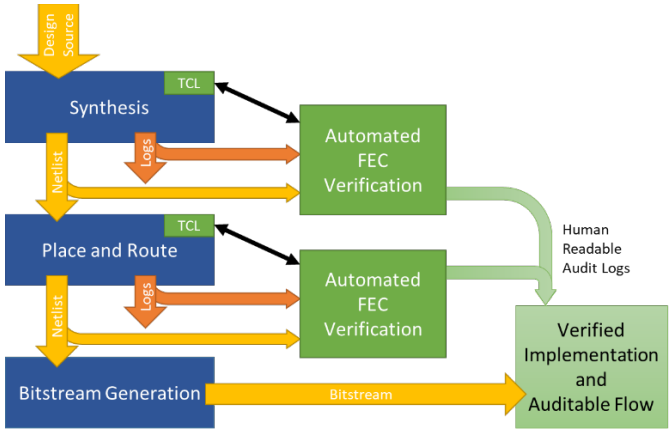


Fig. 2. Automated FEC verification integrated with FPGA build flow

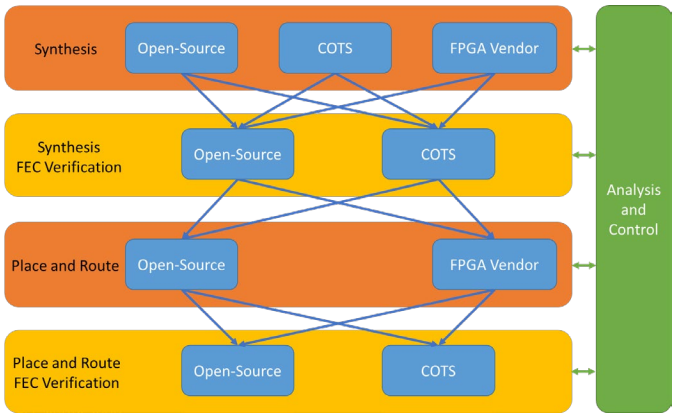


Fig. 3. FPGA EDA Tool Verification Framework

TABLE I. REGISTER OPTIMIZATION IDENTIFICATION AND HINT GENERATION

| Optimization | Vivado Analysis | Synplify Analysis | Formality Hints | SymbiYosys Hints |
|---|---|---|---|---|
| Unused Register Removal | Log | Log | guide_reg_removal | — |
| Register Merging | Log | Log | guide_reg_merging | Add compare point(s) |
| Register Replication (Synthesis) | Netlist | Log and Netlist | guide_reg_duplication | Add compare point(s) |
| Register Replication (Implementation) | Log | n/a | guide_reg_duplication | Add compare point(s) |
| Register Retiming (Synthesis) | Log | Netlist | set_parameters -retimed | — |
| Register Retiming (Implementation) | Log | n/a | set_parameters -retimed | — |
| FSM State Register Re-encoding | Log | Log | guide_fsm_reencoding | — |

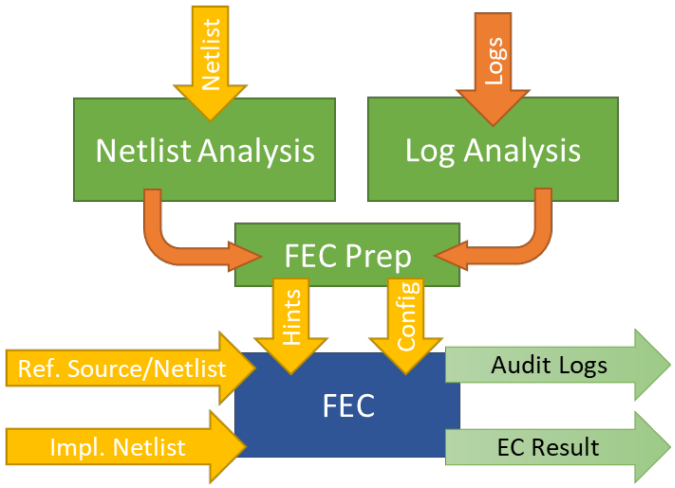


Fig. 4. Automated FEC Verification

### A. Log and Netlist Analysis

Our tool includes Python functions that automatically identify optimizations performed by EDA tools by analyzing the generated log files and netlists. Many types of optimizations are reported in the logs, such as the FSM re-encoding shown in Fig. 5. Other optimizations are not reported in the logs and require netlist analysis for identification. The netlist analysis techniques used vary depending on the synthesis tool and optimization, but can include name matching (e.g., a *_repl* at the end of a register indicating it is a replicated register) or searching netlist cell attributes for useful annotations. Log parsing and netlist analysis are supported for both Xilinx Vivado and Synopsys Synplify.

```
-----------------------------------------------
  State |  New Encoding |  Previous Encoding
-----------------------------------------------
     S0 |           001 |                  00
     S1 |           010 |                  01
     S2 |           100 |                  10
-----------------------------------------------
INFO: [Synth 8-3354] encoded FSM with state register
'state_reg' using encoding 'one-hot' in module 'top'
```

Fig. 5. Vivado log for FSM re-encoding

```
guide_fsm_reencoding -design top
 -previous_state_vector {state_reg[1] state_reg[0]}
 -current_state_vector { FSM_onehot_state_reg[2]
   FSM_onehot_state_reg[1] FSM_onehot_state_reg[0]}
 -state_reencoding { { S0 2#00 2#001 }
   { S1 2#01 2#010 } { S2 2#10 2#100 } }
```

Fig. 6. Automatically-generated Formality hint for FSM re-encoding

### B. FEC Automation Using Synopsys Formality

Synopsys Formality is a combinational logic equivalency checking tool designed for use with ASIC designs. Because it does not support sequential formal analysis, Formality requires compare points to be specified for every register in both the reference and implementation designs. The tool includes mechanisms to identify matching registers in the two designs based on the names and structure of elements in the circuit. However, for EDA optimizations that add, remove, or relocate registers, Formality requires the user to provide guidance, or hints, to identify the appropriate compare points. These hints are specified using proprietary TCL commands. While register optimizations may not be as common in ASIC implementation tools, they are applied extensively by FPGA implementation tools, making the specification of hints for all registers affected by optimizations a significant manual effort.

We have developed functions that utilize the register optimization information extracted from the logs and netlists generated by EDA tools to automatically generate corresponding hints for Formality. TABLE I. shows the complete list of optimizations for which hint generation is supported. An example of a hint that was automatically generated for an FSM state register re-encoding is shown in Fig. 6. Without this hint, FEC fails because Formality is unable to properly compare the two state registers in the reference design with the three state registers in the implementation design. Our automation eliminates the manual effort to analyze such FEC failures and write appropriate hints. Our tool also generates a batch script for Formality that includes commands to read the reference and implementation designs, the Xilinx primitive models, the generated hints, and then to execute the FEC operation. If the verification fails, a Formality command will be called to analyze the failure(s) and provide information on which logic cones are different for further analysis.

### C. FEC Automation Using Open-source Tools

We have implemented an FEC methodology using a collection of open-source tools. Our framework generates a script that uses Yosys [2] to read the reference and implementation designs as well as the Xilinx primitive models and combine them into a Verilog miter circuit. Next, Yosys is used to generate a formal model of the miter circuit in SMT-LIBv2 format [3], which is supported by several open-source Satisfiability Modulo Theories (SMT) solvers. Then, SymbiYosys [4] is used to launch one or more SMT solvers to look for input sequences that violate equivalence assertions; if a violation is found, a Verilog testbench and VCD waveform file are generated to demonstrate the difference. The user can easily specify which SMT solver(s) to be used, which is important given the wide variation in runtime observed in our testing (see Sec IV).

TABLE II. TEST DESIGN RESULTS

| | Vivado Synthesis | | | FEC Verification | | | | |
|---|---|---|---|---|---|---|---|---|
| **Design** | **LUTs** | **FFs** | **Time (s)** | **Generated Hints** | **Compare Points** | **Z3 Time (s)** | **Yices2 Time (s)** | **Bitwuzla Time (s)** |
| SERV RISC-V [6] | 201 | 165 | 50 | 10 | 298 | 3736 | 41 | 5 |
| AES GoogleVault [7] | 5419 | 2717 | 69 | 3 | 2846 | 59397 | 92 | 60 |
| Keccak SHA3 [8] | 7647 | 2212 | 245 | 0 | 2726 | 17 | 102 | 93 |

Unlike Formality, this method utilizes Bounded Model Checking (BMC) that supports sequential analysis. This means that register optimizations can be handled without user-generated hints. However, this leads to long runtimes for deeply-pipelined designs. To address this, we developed a Yosys plugin with the Pyosys [5] API that uses log and netlist analysis to identify compare points and encode them into the miter circuit; this shortens sequential paths, which reduces the state space that needs to be evaluated by the SMT solver and results in runtime reduction.

## IV. RESULTS

The automated FEC verification methodology was applied to three open-source test designs: a RISC-V core [6], an AES encryption core [7], and a SHA3 hashing core [8]. The runtimes to synthesize each core using Xilinx Vivado 2019.2 as well as the numbers of LUTs and FFs in the post-synthesis netlists are listed in TABLE II. The table also lists the numbers of automatically generated hints and the total numbers of compare points used for FEC analysis. For each design, SymbiYosys was used to perform FEC between the HDL source code and the post-synthesis netlist. Each FEC comparison was conducted three times using different SMT solver implementations (Z3 [9], Yices2 [10], and Bitwuzla [11]); all three solvers reported equivalence, but the runtimes varied significantly as shown in Fig. 7 and TABLE II. These results show that FEC analysis performance is highly dependent on the structure of a specific design. For example, Z3 was significantly slower than the other two solvers for both the RISC-V and AES cores, but it was the fastest solver on the SHA3 core. This highlights a key benefit of our flexible framework that uses the open SMT-LIBv2 interface to support multiple formal solver implementations.

## V. CONCLUSIONS AND FUTURE WORK

We have demonstrated the ability to automate and integrate formal equivalency checking for FPGAs into multiple EDA tool flows to provide assurance that the EDA tools are trustworthy.

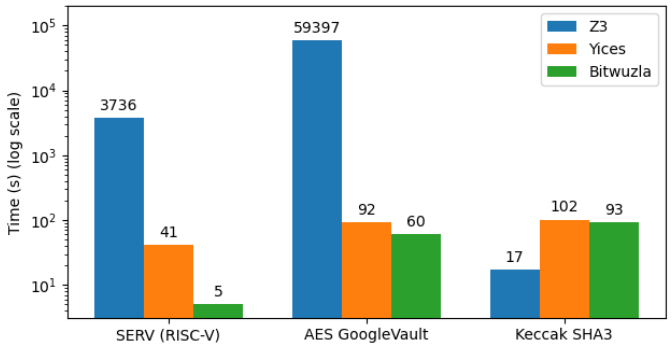


Fig. 7. SymbiYosys FEC runtime for multiple design and solver combinations

Our developed framework supports both commercial and open-source EDA and verification tools, and is designed to be extensible to support other tools and device families in the future. Our log and netlist analysis functions are able to identify all types of register optimizations and provide the necessary guidance for successful FEC verification.

This tool may be used to improve trust and assurance of a design and the EDA flow. Future areas of enhancement include support for automated verification of inferred hard IP primitives (e.g., BRAMs and DSPs) and expanded compare point identification in the open-source method to further reduce runtime.

## REFERENCES


[1] O. Kindgren, "A scalable approach to IP management with FuseSoC," in *Workshop Open Source Design Automation*, 2019.

[2] C. Wolf, "Yosys Open SYnthesis Suite." http://www.yosyshq.net/yosys/

[3] C. Barrett, A. Stump, and C. Tinelli, "The SMT-LIB Standard: Version 2.0," in *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories (Edinburgh, UK)*, 2010.

[4] C. Wolf, "SymbiYosys (sby) -- Front-end for Yosys-based formal verification flows." https://github.com/YosysHQ/sby

[5] B. Tutzer, C. Krieg, C. Wolf, and A. Jantsch, "Python Wraps Yosys for Rapid Open-Source EDA Application Development," in *Workshop Open Source Design Automation*, 2019.

[6] "SERV - The SErial RISC-V CPU." https://github.com/olofk/serv (accessed Jan. 01, 2023).

[7] "Google Project Vault AES." https://github.com/ProjectVault/orp/tree/master/hardware/mselSoC/src/systems/geophyte/rtl/verilog/crypto_aes/rtl/verilog (accessed Jan. 01, 2023).

[8] "SHA3 (KECCAK)." https://opencores.org/projects/sha3 (accessed Jan. 01, 2023).

[9] L. de Moura and N. Bjørner, "Z3: An efficient SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008, vol. 4963 LNCS, pp. 337–340.

[10] B. Dutertre, "Yices 2.2," in *Computer-Aided Verification (CAV)*, 2014, vol. 8559 LNCS, pp. 737–744.

[11] A. Niemetz and M. Preiner, "Bitwuzla at the SMT-COMP 2020," in *18th International Workshop on Satisfiability Modulo Theories*, May 2020.